

ROS2 Doctor

PHẦN 1: PROJECT BRIEF

1. Domain

Robot Field Operations — Post-run Diagnostics trên nền tảng ROS2, tập trung vào AMR (Autonomous Mobile Robot) sử dụng Nav2 stack.

Đây là domain vận hành robot sau khi đã triển khai thực tế. Kỹ sư phải đối mặt với câu hỏi hàng ngày: "Robot vừa chạy xong ca, nó bị lỗi/hành vi lạ — chuyện gì đã xảy ra?" Câu trả lời nằm trong rosbag, nhưng đọc và phân tích rosbag là công việc thủ công, chậm, lặp đi lặp lại, và phụ thuộc nặng vào kinh nghiệm cá nhân.

2. Người dùng mục tiêu

Kỹ sư robotics đang vận hành AMR thực tế (nhà kho, nhà máy, giao hàng, xe tự hành giai đoạn testing). Đặc điểm chung: mỗi ngày đều phải debug robot qua rosbag — đây là phần tốn thời gian nhất trong workflow.

3. Pain Points

3.1. Tồn thời gian

Thời gian đọc và phân tích thủ công: 1–4 giờ/bag. Kỹ sư phải mở 4–5 tool rời rạc (rqt_bag, rviz, plotjuggler, terminal grep log) để xem từng khía cạnh — không có cái nhìn tổng quan nào trong một nơi duy nhất.

3.2. Phụ thuộc kinh nghiệm

Junior engineer không biết cái gì là bất thường (frequency bao nhiêu là đúng, TF gây ở đâu, timestamp lệch bao nhiêu là vấn đề). Kiến thức debug là tribal knowledge — truyền miệng trong team, không được document hóa.

3.3. Không có giải thích & đề xuất

Biết lỗi rồi nhưng không biết fix thế nào: Node nào chịu trách nhiệm? Nguyên nhân gốc là gì? Sửa launch file hay code node? Không có tool nào trả lời chuỗi câu hỏi này tự động.

4. Giải pháp

Xây dựng pipeline tự động phân tích rosbag: phát hiện bất thường bằng rule-based analyzer, sau đó dùng vLLM self-hosted (long context + tool calling) để giải thích nguyên nhân gốc rễ bằng ngôn ngữ tự nhiên và đề xuất hướng khắc phục cụ thể. Kỹ sư xem kết quả qua dashboard và duyệt/reject chẩn đoán (HITL).

5. Phạm vi (Scope)

Trong phạm vi	Ngoài phạm vi
AMR chạy Nav2 stack	Robot cánh tay, drone, xe tự hành phức tạp
Phân tích rosbag post-run	Monitoring real-time robot đang chạy
6 loại lỗi phổ biến Nav2	Lỗi đặc thù perception/SLAM
Chẩn đoán + đề xuất fix	Tự động fix/can thiệp hệ thống

6. Tech Stack

Thành phần	Công nghệ	Lý do chọn
Thu data	Gazebo + Nav2 + DVC + Google Drive	Simulation có control, DVC quản lý version dataset
Parse bag	rosbags + numpy	Nhẹ hơn rosbag2_py, vectorized nhanh hơn pandas
Phân tích	Polars	Đa luồng, xử lý hàng triệu dòng trong giây, thay thế pandas
LLM inference	vLLM (Qwen2.5 / Llama3)	Self-hosted, long context 128K, tool calling tốt
Đánh giá	DeepEval	LLM-as-judge cho chất lượng giải thích
Backend	FastAPI	Async, streaming, expose 4 khu vực API rõ ràng
Frontend	React / Next.js	Dashboard timeline + HITL + chat
Deploy	Docker	Đóng gói toàn bộ pipeline

PHẦN 2: PRODUCT REQUIREMENTS DOCUMENT (PRD)

1. Kiến trúc & Luồng triển khai

Bước 0: Thu & Quản lý Dataset

- Chạy Gazebo + Nav2 với TurtleBot3 (model waffle)
- Record rosbag (.mcap): 10 bag healthy + 30 bag lỗi (5 bag × 6 loại) + 10 bag lỗi kết hợp
- Quản lý dataset bằng DVC, lưu trữ trên Google Drive — team tải về bằng dvc pull
- Mỗi bag lỗi kèm ground truth JSON: loại lỗi, thời điểm inject, anomaly kỳ vọng

Tầng 0: Data Salvage

- Thử đọc bag theo phương thức chuẩn (rosbags library)
- Nếu bag bị hỏng (node crash giữa chừng, mất nguồn đột ngột): chạy binary scan + recovery để khôi phục message còn nguyên vẹn
- Output: Valid bag hoặc Partial bag — luôn có dữ liệu đầu vào cho tầng sau

Tầng 1: Denormalization & Analytics

- Denormalizer (rosbags + numpy): parse .mcap → structured array → tính toán vectorized. Flatten ROS message dạng nested thành columnar data.
- Polars Analytics: tính frequency, timestamp delta, phát hiện NaN, missing data — xử lý đa luồng, nhanh hơn pandas 5-10x
- Rule-based detect: phát hiện gap tần số, timestamp drift, node im lặng, TF gãy, NaN sensor
- Scrubber (PII removal): lọc GPS coordinates, IP, credential trước khi gửi LLM
- Output: Anomaly JSON (vài KB) + In-Memory Polars DataFrames

Tầng 2A: vLLM Agent

- Host vLLM self-hosted với model long context + tool calling tốt (Qwen2.5-72B hoặc Llama3.1-70B)
- Nhận Anomaly JSON → suy luận → tự gọi tool khi cần thêm data → tiếp tục suy luận → Root-Cause Analysis
- Không dùng LangGraph — vLLM tự handle tool calling natively, đơn giản hơn

Tầng 2B: Polars Tool Gateway

- `check_tf_tree_gap(start_time, end_time)`: kiểm tra TF tree có bị gãy trong khoảng thời gian
- `query_frequency(topic, time_range)`: lấy frequency thực tế của topic
- `compare_topics(topic_a, topic_b)`: so sánh 2 topic có mất data cùng lúc không
- `get_sample_messages(topic, time_sec, count)`: lấy mẫu message quanh thời điểm lỗi
- Mỗi tool trả kết quả qua Scrubber Gateway trước khi gửi về agent

Tầng 3: FastAPI + React

- FastAPI expose 4 khu vực API: /datasets, /analysis, /review, /dashboard
- React UI — Timeline: hiển thị detection theo trục thời gian, đánh dấu đỏ tại vùng lỗi
- React UI — Human Review: kỹ sư Approve/Reject từng chẩn đoán (HITL)
- React UI — Dashboard: tổng quan nhanh toàn bộ bag (số lỗi, severity, root cause)
- Progressive Loading: hiển thị kết quả ngay khi tầng 1 xong, không chờ LLM

2. Các loại lỗi cần phát hiện

STT	Nhóm	Loại lỗi	Mô tả
1	Frequency	Topic dead	Topic ngừng publish hoàn toàn — node crash hoặc QoS mismatch
2	Frequency	Frequency drop	Tần số thực tế < 50% kỳ vọng — CPU overload, hardware issue
3	Frequency	Burst bất thường	Tần số đột ngột cao bất thường — buffer flush, driver bug
4	Timestamp	Clock drift	Delta giữa sensor tăng dần — thiếu NTP/PTP sync
5	Timestamp	Timestamp jump/lùi	Timestamp nhảy đột ngột hoặc đi lùi — clock reset, node restart
6	TF	TF gap	Chuỗi map→odom→base_link bị đứt trong khoảng thời gian
7	TF	TF conflict	2 node cùng publish một transform — cấu hình launch sai
8	TF	TF loop	Cycle trong TF tree — URDF lỗi
9	Node	Node crash	Tất cả topic cùng namespace mất đồng thời — suy luận node crash
10	Node	Node restart	Gap rồi phục hồi — respawn hoặc supervisor restart
11	Data quality	NaN / Inf	Giá trị không hợp lệ trong sensor — hardware lỗi, driver bug
12	Data quality	Out of range	Giá trị ngoài physical range hợp lệ — encoding error
13	QoS	Message gap	Khoảng trống lớn bất thường giữa các message
14	QoS	QoS mismatch	Publisher BEST_EFFORT vs subscriber RELIABLE — mất message thâm lặng

3. Dataset

3.1. Cấu trúc dataset

Loại	Số bag	Mục đích
Healthy (baseline)	10	Analyzer phải nói "không có lỗi" — test false positive
Mỗi loại lỗi đơn (6 loại)	$5 \times 6 = 30$	Test detection từng loại lỗi
Lỗi kết hợp (2-3 lỗi cùng lúc)	10	Test correlation + root cause analysis
Tổng	~50 bag	Đủ cho MVP benchmark

3.2. Topics được record (8 topic cốt lõi)

Topic	Nhóm	Detect lỗi
-------	------	------------

/tf, /tf_static	Transform	TF gap, broken chain, conflict
/scan	Sensor	Frequency drop, topic dead
/odom	Robot State	Timestamp lệch, odometry drift
/imu	Sensor	NaN, sensor freeze, clock drift
/cmd_vel	Command	Node crash (velocity mất đột ngột)
/amcl_pose	State Estimate	Mất định vị
/diagnostics	Health	System health tổng quan

3.3. Quản lý với DVC

- Khởi tạo: `dvc init` trong repo
- Track data: `dvc add bags/` → `commit .dvc` file lên Git
- Remote storage: `dvc remote add -d myremote gdrive://<folder_id>`
- Team tải về: `dvc pull` — đảm bảo reproducible trên mọi máy
- Version control: mỗi lần thêm bag mới → `dvc add` + `git commit` → có tag version

4. Yêu cầu chức năng

4.1. Cơ bản

- Tool nạp rosbag mẫu, hiển thị danh sách topic/tần số/timeline
- Tự phát hiện các bất thường cơ bản và báo cáo
- Phân vai trò: ≥ 2 vai trò (Engineer + Reviewer)
- HITL: kỹ sư duyệt kết luận

4.2. Nâng cao

- Agent chẩn đoán đa lỗi, lý giải nguyên nhân, đề xuất khắc phục
- Kiểm tra TF và đồng bộ thời gian
- Benchmark độ chính xác phát hiện trên bộ bag có lỗi đã biết
- Tối ưu thời gian/chi phí phân tích

5. Yêu cầu phi chức năng

5.1. Bảo mật

- File upload qua sandbox container (cô lập, không có network)
- Giới hạn file size (max 2GB)
- Chỉ parse known message types
- Scrubber lọc GPS coordinates, IP, credentials trước khi gửi LLM
- Xóa file bag gốc sau phân tích, chỉ giữ anomaly report

5.2. Hiệu năng

- Tầng 1 (Polars) xử lý bag vài GB trong vài giây
- Tầng 2 (vLLM) chẩn đoán trong vài phút

- Mục tiêu: thủ công 1–4 giờ → pipeline < 10 phút
- Đo token cost và latency mỗi lần chẩn đoán

5.3. Ràng buộc

- Agent chỉ chẩn đoán và gợi ý, KHÔNG tự động can thiệp hệ thống robot đang chạy
- Phân tích trên log ghi sẵn (post-run), không monitoring real-time
- Hiệu quả đo được: precision, recall, thời gian chẩn đoán, chi phí token

6. Benchmark & Đánh giá

Hạng mục	Metric	Mục tiêu
Tầng 1 — Detection	Precision / Recall	> 80% precision, > 75% recall trên 50 bag
Tầng 1 — Timing	Time accuracy	Sai lệch thời điểm lỗi < 2 giây so với ground truth
Tầng 2 — LLM quality	DeepEval (LLM-as-judge)	Giải thích đúng nguyên nhân, đề xuất actionable, không hallucinate
Tầng 2 — Root cause	Root cause accuracy	> 70% chỉ đúng nguyên nhân gốc
Hiệu năng	Thời gian tổng	< 10 phút/bag (so với 1–4 giờ thủ công)

7. MVP Prioritization

Tính năng	Competitor (rqt_bag/plotjuggler)	ROS2 Doctor — Cơ bản	ROS2 Doctor — Nâng cao
Must-Haves			
Đọc file .db3/.mcap	Y	Y	Y
Hiển thị topic + timeline	Y	Y	Y
Performance Benefits			
Tự động phát hiện lỗi	Thấp (thủ công)	Cao	Cao
Giải thích nguyên nhân NLP	Không	Có	Có (sâu hơn)
Thời gian phân tích	1–4 giờ	< 10 phút	< 5 phút
Delighters			
vLLM self-hosted		Y	Y
HITL Approve/Reject		Y	Y
DVC dataset versioning		Y	Y
Data Salvage (bag corrupt)			Y
Benchmark accuracy			Y

PHẦN 3: TÀI LIỆU THAM KHẢO

Mentor Duty 1/8

Trong quá trình robot định vị, môi trường xung quanh không phải lúc nào cũng tĩnh. Khi có vật thể động (người đi qua, xe đẩy, hàng hóa được di chuyển) xuất hiện gần robot — đặc biệt gần LiDAR — sẽ gây ra các vấn đề:

- Mất tia (ray missing): vật thể chắn một phần tia laser → LiDAR không quét được vùng phía sau vật thể → dữ liệu /scan bị thiếu
- Đọc sai khoảng cách: LiDAR đo được vật thể động thay vì tường/vật cản cố định → AMCL so sánh với map tĩnh thấy không khớp → định vị bị nhiễu
- Định vị trôi: nếu vật thể động xuất hiện liên tục → AMCL liên tục bị nhiễu → particle cloud phân tán → robot không chắc chắn về vị trí của mình

Repo tham khảo

<https://github.com/vikramnagashoka/rosbag-resurrector>